

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Тульский государственный университет»

Институт прикладной математики и компьютерных наук

Кафедра вычислительной техники

Отчет по лабораторной работе №8

На тему «Приемочное тестирование и BDD (Behavior-Driven Development)»

По дисциплине «Процессы обеспечения качества программного
обеспечения»

Выполнил

_____ ст. гр. 220621: Забродин И. В.

Проверил

_____ ст. п. кафедры ВТ, к. т. н.: Пестин М. С.

Тула 2026

Цель работы

Изучить методологию приемочного тестирования, освоить подход Behavior-Driven Development (BDD) и инструменты для спецификации требований на понятном заказчику языке. Получить практические навыки написания сценариев на языке Gherkin и их автоматизации с использованием специализированных фреймворков.

Ход работы

Для копии решения из предыдущей лабораторной работы создал проект BDDTests после удаления всех проектов кроме Core и WinForms, содержащих логику регистрации и авторизации проекта (Рисунок 1)

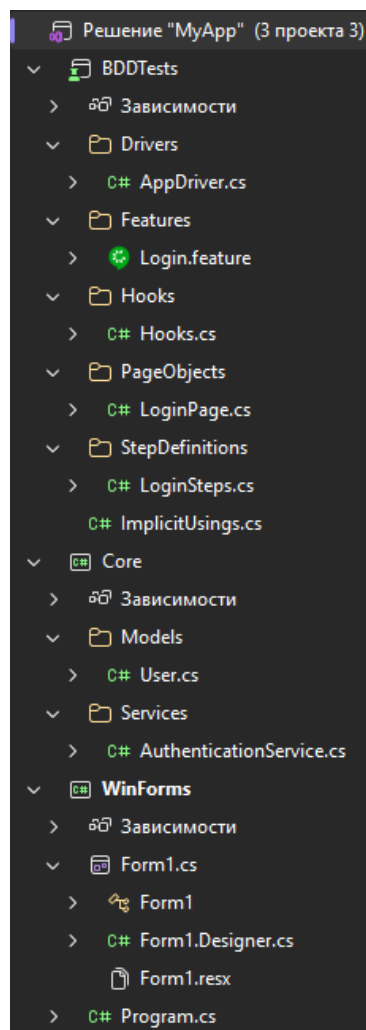


Рисунок 1 – Структура проекта для данной лабораторной работы

Для автоматизации приемочных сценариев был выбран Reqnroll — современный форк SpecFlow, полностью совместимый с .NET 10 и активно развивающийся. Основные причины выбора:

Интеграция с .NET — Reqnroll написан на C# и легко интегрируется с существующим проектом, использующим WinForms и библиотеку Core.

Поддержка Gherkin — позволяет писать сценарии на понятном бизнес-языке, что соответствует методологии BDD.

Гибкость — может работать с любым тестовым раннером (MSTest, NUnit, xUnit); мы использовали MSTest для единообразия с предыдущими лабораторными.

Активная поддержка — в отличие от устаревшего SpecFlow, Reqnroll обновляется под новые версии .NET и не имеет проблем с совместимостью.

Сообщество и документация — наличие подробных руководств и примеров упрощает процесс написания step definitions.

Альтернативы (SpecFlow, Cucumber) либо не поддерживают .NET 10, либо требуют дополнительных настроек. Reqnroll позволил быстро создать feature-файлы и связать их с кодом на C#.

Написал Gherkin-сценарии (*Features/Login.features*):

```
Feature: Авторизация пользователей
  Как зарегистрированный пользователь
  Я хочу иметь возможность войти в систему
  Чтобы получить доступ к защищенным функциям

@positive
Scenario: Успешный вход с корректными данными
  Given приложение запущено
  And пользователь с именем "testuser" и паролем "password123" зарегистрирован
  When пользователь вводит логин "testuser" и пароль "password123"
  And нажимает кнопку входа
  Then система показывает сообщение "Успешный вход!"

@negative
Scenario: Вход с неверным паролем
  Given приложение запущено
  And пользователь с именем "testuser" и паролем "password123" зарегистрирован
  When пользователь вводит логин "testuser" и пароль "wrongpass"
  And нажимает кнопку входа
  Then система показывает сообщение "Ошибка: неверный логин или пароль."

@positive
Scenario: Регистрация нового пользователя
  Given приложение запущено
  When пользователь вводит логин "newuser" и пароль "newpass123"
  And нажимает кнопку регистрации
  Then система показывает сообщение "Регистрация успешна! Теперь войдите."
  And пользователь может войти с логином "newuser" и паролем "newpass123"
```

```
@negative
Scenario Outline: Регистрация с некорректными данными
    Given приложение запущено
    And пользователь с именем "existing" и паролем "pass" зарегистрирован
    When пользователь вводит логин "<username>" и пароль "<password>"
    And нажимает кнопку регистрации
    Then система показывает сообщение об ошибке "<error>"
```

```
Examples:
    | username | password | error |
    |          | pass    | Ошибка: заполните все поля |
    | user     |         | Ошибка: заполните все поля |
    | existing | pass    | Ошибка: пользователь уже существует. |
```

```
@positive
Scenario Outline: Проверка политики надежности пароля (IsPasswordStrong)
    Given сервис аутентификации
    When проверяется пароль "<password>"
    Then результат должен быть <expected>
```

```
Examples:
    | password | expected |
    | "short"  | False    |
    | "longenough" | False    |
    | "StrongPass1" | True     |
```

Реализовал программу с выполнением сценариев
(*StepDefinitions/LoginSteps.cs*):

```
using FluentAssertions;
using MyApp.BDDTests.Drivers;
using MyApp.Core.Services;

namespace MyApp.BDDTests.StepDefinitions;

[Binding]
public class LoginSteps
{
    private readonly AppDriver _driver;
    private readonly ScenarioContext _scenarioContext;
    private AuthenticationService? _authService;
    private bool? _passwordCheckResult;
    public LoginSteps(AppDriver driver, ScenarioContext scenarioContext)
    {
        _driver = driver;
        _scenarioContext = scenarioContext;
    }

    [Given(@"приложение запущено")]
    public void GivenПриложениеЗапущено()
    {
        Console.WriteLine("Application is running");
    }

    [Given(@"пользователь с именем ""(.*)"" и паролем ""(.*)"" зарегистрирован")]
    public void GivenПользовательЗарегистрирован(string username, string password)
    {
        _driver.LoginPage.RegisterUser(username, password);

        var message = _driver.LoginPage.GetMessageText();
        if (!message.Contains("Регистрация успешна"))
        {
            throw new Exception($"Не удалось зарегистрировать пользователя: {message}");
        }

        _driver.LoginPage.ClearFields();

        Console.WriteLine($"User {username} registered via UI");
    }

    [When(@"пользователь вводит логин ""(.*)"" и пароль ""(.*)""")]
    public void WhenПользовательВводитЛогинИПароль(string username, string password)
    {
        _driver.LoginPage.ClearFields();
```

```

        _driver.LoginPage.EnterUsername(username);
        _driver.LoginPage.EnterPassword(password);
        Console.WriteLine($"Entered credentials: '{username}' / '{password}'");
    }

    [When(@"нажимает кнопку входа")]
    public void WhenНажимаетКнопкуВхода()
    {
        _driver.LoginPage.ClickLogin();
        Thread.Sleep(500);
        Console.WriteLine("Clicked Login button");
    }

    [When(@"нажимает кнопку регистрации")]
    public void WhenНажимаетКнопкуРегистрации()
    {
        _driver.LoginPage.ClickRegister();
        Thread.Sleep(500);
        Console.WriteLine("Clicked Register button");
    }

    [Then(@"система показывает сообщение ""(.*)""")]
    public void ThenСистемаПоказываетСообщение(string expectedMessage)
    {
        var actual = _driver.LoginPage.GetMessageText();
        Console.WriteLine($"Expected: '{expectedMessage}', Actual: '{actual}'");
        actual.Should().Be(expectedMessage);
    }

    [Then(@"система показывает сообщение об ошибке ""(.*)""")]
    public void ThenСистемаПоказываетСообщениеОбОшибке(string expectedError)
    {
        var actual = _driver.LoginPage.GetMessageText();
        Console.WriteLine($"Expected error: '{expectedError}', Actual: '{actual}'");
        actual.Should().Be(expectedError);
    }

    [Then(@"пользователь может войти с логином ""(.*)"" и паролем ""(.*)""")]
    public void ThenПользовательМожетВойти(string username, string password)
    {
        _driver.LoginPage.ClearFields();
        _driver.LoginPage.EnterUsername(username);
        _driver.LoginPage.EnterPassword(password);
        _driver.LoginPage.ClickLogin();
        Thread.Sleep(500);

        var message = _driver.LoginPage.GetMessageText();
        Console.WriteLine($"Login after registration result: '{message}'");
        message.Should().Be("Успешный вход!");
    }

    [Given(@"сервис аутентификации")]
    public void GivenСервисАутентификации()
    {
        // Используем конструктор по умолчанию
        _authService = new AuthenticationService();
        Console.WriteLine("Authentication service created");
    }

    [When(@"проверяется пароль ""(.*)""")]
    public void WhenПроверяетсяПароль(string password)
    {
        Console.WriteLine($"Checking password: '{password}'");
        _passwordCheckResult = _authService!.IsPasswordStrong(password);
        Console.WriteLine($"Result: {_passwordCheckResult}");
    }

    [Then(@"результат должен быть (.*)")]
    public void ThenРезультатДолженБыть(bool expected)
    {
        Console.WriteLine($"Expected: {expected}, Actual: {_passwordCheckResult}");
        _passwordCheckResult.Should().Be(expected);
    }
}

```

Перед и после выполнения каждого теста мы можем выполнять определённые действия, которые описаны в *Hooks/Hooks.cs*:

```

using MyApp.BDDTests.Drivers;
using Reqnroll.BoDi;

namespace MyApp.BDDTests.Hooks;

[Binding]
public class Hooks
{
    private readonly IObjectContainer _objectContainer;

    public Hooks(IObjectContainer objectContainer)
    {
        _objectContainer = objectContainer;
    }

    [BeforeScenario]
    public void RegisterDriver()
    {
        CleanupUserFile();

        var driver = new AppDriver();
        _objectContainer.RegisterInstanceAs(driver);
    }

    [AfterScenario]
    public void Cleanup()
    {
        var driver = _objectContainer.Resolve<AppDriver>();
        driver.Dispose();
    }

    private void CleanupUserFile()
    {
        var appData = Path.Combine(
            Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData),
            "MyApp", "users.json");

        if (File.Exists(appData))
        {
            File.Delete(appData);
            Console.WriteLine($"Deleted user file: {appData}");
        }
        else
            Console.WriteLine($"No user file found at: {appData}");
    }
}

```

Эти хуки и реализация логики сценариев используют для своей работы драйвер для работы с нашим WinForms приложением. Реализован в *Drivers/AppDriver.cs*:

```

using MyApp.BDDTests.PageObjects;

namespace MyApp.BDDTests.Drivers;

public class AppDriver : IDisposable
{
    public LoginPage LoginPage { get; private set; }

    public AppDriver()
    {
        LoginPage = new LoginPage();
        LoginPage.Launch();
    }

    public void Dispose()
    {
        LoginPage.Dispose();
    }
}

```

На данный момент наше приложение содержит лишь одну страницу — LoginPage, которую использует вышеуказанный драйвер приложения (*PageObjects/LoginPage.cs*):

```
using FlaUI.Core.AutomationElements;
using FlaUI.UIA3;

namespace MyApp.BDDTests.PageObjects;

public class LoginPage : IDisposable
{
    private FlaUI.Core.Application? _app;
    private Window? _window;
    private UIA3Automation? _automation;

    public void Launch()
    {
        var baseDir = AppDomain.CurrentDomain.BaseDirectory;
        var solutionDir = Path.GetFullPath(Path.Combine(baseDir, @"..\\..\\..\\..\\"));
        var appPath = Path.Combine(solutionDir, "WinForms", "bin", "Debug", "net10.0-windows",
"WinForms.exe");

        _app = FlaUI.Core.Application.Launch(appPath);
        _automation = new UIA3Automation();
        _window = _app.GetMainWindow(_automation, TimeSpan.FromSeconds(10));
    }

    public void Close()
    {
        _app?.Close();
        _automation?.Dispose();
    }

    public void Dispose()
    {
        Close();
    }

    private FlaUI.Core.AutomationElements.TextBox UsernameTextBox =>
        _window!.FindFirstDescendant(cf => cf.ByAutomationId("txtUsername")).AsTextBox();

    private FlaUI.Core.AutomationElements.TextBox PasswordTextBox =>
        _window!.FindFirstDescendant(cf => cf.ByAutomationId("txtPassword")).AsTextBox();

    private FlaUI.Core.AutomationElements.Button LoginButton =>
        _window!.FindFirstDescendant(cf => cf.ByAutomationId("btnLogin")).AsButton();

    private FlaUI.Core.AutomationElements.Button RegisterButton =>
        _window!.FindFirstDescendant(cf => cf.ByAutomationId("btnRegister")).AsButton();

    private FlaUI.Core.AutomationElements.Label MessageLabel =>
        _window!.FindFirstDescendant(cf => cf.ByAutomationId("lblMessage")).AsLabel();

    public void EnterUsername(string username) => UsernameTextBox.Text = username;
    public void EnterPassword(string password) => PasswordTextBox.Text = password;
    public void ClickLogin() => LoginButton.Invoke();
    public void ClickRegister() => RegisterButton.Invoke();
    public string GetMessageText() => MessageLabel.Text;

    public void LoginAs(string username, string password)
    {
        EnterUsername(username);
        EnterPassword(password);
        ClickLogin();
        Wait(500);
    }

    public void RegisterUser(string username, string password)
    {
        EnterUsername(username);
        EnterPassword(password);
        ClickRegister();
        Wait(500);
    }
}
```

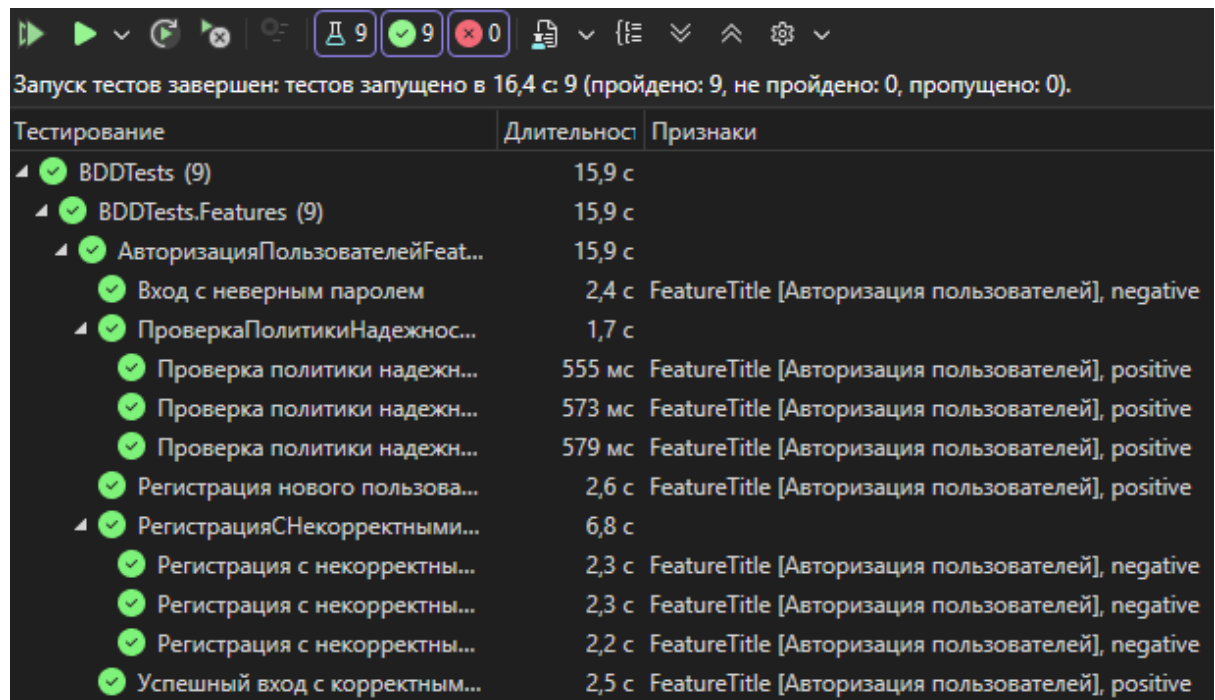
```

public void ClearFields()
{
    UsernameTextBox.Text = "";
    PasswordTextBox.Text = "";
}

private void Wait(int ms) => Thread.Sleep(ms);
}

```

Все тесты для реализованной в предыдущей лабораторной работы логики отработали успешно (Рисунок 2):



Тестирование	Длительность	Признаки
▲ ✓ BDDTests (9)	15,9 с	
▲ ✓ BDDTests.Features (9)	15,9 с	
▲ ✓ АвторизацияПользователейFeat...	15,9 с	
✓ Вход с неверным паролем	2,4 с	FeatureTitle [Авторизация пользователей], negative
▲ ✓ ПроверкаПолитикиНадежнос...	1,7 с	
✓ Проверка политики надежн...	555 мс	FeatureTitle [Авторизация пользователей], positive
✓ Проверка политики надежн...	573 мс	FeatureTitle [Авторизация пользователей], positive
✓ Проверка политики надежн...	579 мс	FeatureTitle [Авторизация пользователей], positive
✓ Регистрация нового пользова...	2,6 с	FeatureTitle [Авторизация пользователей], positive
▲ ✓ РегистрацияСНекорректными...	6,8 с	
✓ Регистрация с некорректны...	2,3 с	FeatureTitle [Авторизация пользователей], negative
✓ Регистрация с некорректны...	2,3 с	FeatureTitle [Авторизация пользователей], negative
✓ Регистрация с некорректны...	2,2 с	FeatureTitle [Авторизация пользователей], negative
✓ Успешный вход с корректным...	2,5 с	FeatureTitle [Авторизация пользователей], positive

Рисунок 2 – Результат работы тестов

Вывод

В ходе лабораторной работы была изучена методология приемочного тестирования и Behavior-Driven Development (BDD). На практике освоено:

Написание сценариев на языке Gherkin с использованием ключевых слов Feature, Scenario, Given, When, Then, And, а также Scenario Outline с таблицей Examples для параметризации.

Автоматизация сценариев с помощью Reqnroll: создание step definitions, связывающих Gherkin-шаги с кодом на C#, использующим UI-автоматизацию (FlaUI) и сервисы приложения.

Организация независимости тестов через очистку состояния (удаление файла users.json и перезапуск приложения перед каждым сценарием).

Сравнение с предыдущими подходами (модульное тестирование, UI-тесты без BDD):

Gherkin-сценарии оказались значительно более читаемыми и понятными даже для человека, не знакомого с кодом. Они выступают в роли "живой документации", объединяющей требования и тесты.

Процесс написания тестов через BDD заставляет сначала продумать поведение системы с точки зрения пользователя, что улучшает понимание требований.

Поддержка сценариев при изменении функциональности упрощается, так как изменения вносятся либо в feature-файлы, либо в соответствующие step definitions, но логика тестов остаётся прозрачной.

Оценка понятности Gherkin-сценариев для заказчика – сценарии действительно могут быть поняты не программистом, так как они написаны на естественном языке (русском) и описывают конкретные шаги пользователя и ожидаемые реакции системы. Это способствует лучшему взаимодействию между аналитиками, разработчиками и тестировщиками.

Преимущества BDD:

Единый язык общения в команде.

Тесты = требования, что снижает риск недопонимания.

Возможность автоматизации на разных уровнях (модульном, интеграционном, UI).

Обнаружение неоднозначностей в требованиях на ранних этапах.

Сложности, с которыми столкнулся:

Необходимость тщательной изоляции тестов (очистка состояния приложения между сценариями).

Проблемы с кавычками в таблицах Examples (пустые строки должны быть действительно пустыми, а не содержать "").

Настройка инфраструктуры (Reqnroll + FlaUI) и отладка шагов из-за особенностей UI-автоматизации.

В целом, цель работы достигнута: освоены приёмы приемочного тестирования и BDD, написаны и автоматизированы сценарии для системы авторизации.

Ответы на контрольные вопросы

- 1) Приемочное тестирование — проверка соответствия системы бизнес-требованиям заказчика. От системного отличается тем, что фокусируется на пользовательских сценариях, а не на технической реализации.
- 2) Виды приемочного тестирования: альфа-тестирование (внутри компании), бета-тестирование (реальные пользователи), операционное (OAT), контрактное.
- 3) Критерии приемки (Acceptance Criteria) — условия, которым должно удовлетворять решение. Они ложатся в основу тестовых сценариев.
- 4) BDD (Behavior-Driven Development) — методология, развивающая TDD и ориентированная на поведение системы с точки зрения бизнеса. Использует общий язык для описания требований.
- 5) BDD vs TDD: TDD фокусируется на тестировании кода с технической стороны, BDD — на поведении системы. BDD дополняет TDD, работая на уровне требований.
- 6) Gherkin — язык описания сценариев. Ключевые слова: Feature, Scenario, Given, When, Then, And, But, Scenario Outline, Examples.
- 7) Структура сценария: Given (контекст), When (действие), Then (результат). And/But расширяют шаги.
- 8) Scenario Outline и Examples — параметризованный сценарий, выполняющийся с разными наборами данных из таблицы. Применяется для проверки множества однотипных случаев.
- 9) Основные читатели Gherkin-сценариев — все участники команды (аналитики, разработчики, тестировщики, заказчики). Это обеспечивает общее понимание.
- 10) Единый язык (Ubiquitous Language) — терминология, понятная и бизнесу, и разработчикам, используемая в коде, тестах и обсуждениях.

11) BDD-фреймворки: для .NET: Reqnroll, SpecFlow; для Python: Behave, pytest-bdd; для Java: Cucumber-JVM, JBehave.

12) Step definitions — код, связывающий шаги Gherkin с исполняемыми действиями. Каждый шаг в feature-файле должен иметь соответствующий метод в step definitions.

13) Процесс автоматизации: feature-файл, step definitions, вызов API/UI/сервисов, проверки.

14) BDD для ручного тестирования — да, сценарии могут использоваться как тест-кейсы для ручного прогона.

15) Библиотеки для проверок: FluentAssertions (C#), Shouldly, стандартные Assert.

16) Участники BDD: разработчики, тестировщики, бизнес-аналитики (или заказчики). Их роли: аналитики пишут сценарии, разработчики и тестировщики их автоматизируют.

17) Независимость тестов достигается очисткой состояния (перезапуск приложения, удаление временных файлов, использование транзакций).

18) Автоматизировать все Gherkin-сценарии не обязательно. Некоторые могут оставаться ручными (исследовательское тестирование, сложные UI-сценарии).

19) Частота запуска: автоматизированные приемочные тесты желательно запускать при каждом изменении кода (CI) или хотя бы перед релизом.

20) Метрики: процент прохождения, время выполнения, покрытие требований, количество дефектов, найденных приемочными тестами.